Elements of AIML Lab
Experiment 07 (After-submission Reference): Scaling and One-Hot Encoding

Tofik Ali

March 1, 2026

Contents

1 How to use this reference **1**

2 Scaling (what it does) **1**
2.1 Standardization (z-score) 1
2.2 Min-max normalization 2

3 One-hot encoding (what it does) **2**

4 Worked example (leakage-safe pipeline) **2**
4.1 Step 1: load and split 2
4.2 Step 2: build a ColumnTransformer + Pipeline 2
4.3 Step 3: inspect the transformed feature space (optional but useful) 3

5 What to write in reports (good explanations) **3**

6 Troubleshooting **3**

1 How to use this reference

This reference is meant to be shared **after you submit** Experiment 07. It explains the two most common preprocessing operations in ML:

- scaling numeric features, and
- one-hot encoding categorical features,

while highlighting the most important risk: **data leakage**.

2 Scaling (what it does)

2.1 Standardization (z-score)

Transforms a numeric feature x to:

$$z = \frac{x - \mu}{\sigma}$$

so the transformed feature has mean ≈ 0 and standard deviation ≈ 1 on the training data.

2.2 Min-max normalization

Transforms to:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

so values map to $[0, 1]$ (on the training data range).

3 One-hot encoding (what it does)

A categorical column like `City` with values `{Delhi, Mumbai}` becomes two binary columns such as `City_Delhi`, `City_Mumbai`.

4 Worked example (leakage-safe pipeline)

4.1 Step 1: load and split

```
import pandas as pd
from sklearn.model_selection import train_test_split

df = pd.read_csv("processed.csv") # or your dataset

X = df.drop(columns=["target"]) # change target name
y = df["target"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

4.2 Step 2: build a ColumnTransformer + Pipeline

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LogisticRegression

num_cols = X_train.select_dtypes("number").columns
cat_cols = X_train.select_dtypes(exclude="number").columns

pre = ColumnTransformer([
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
])

pipe = Pipeline([
    ("pre", pre),
    ("clf", LogisticRegression(max_iter=1000)),
])

pipe.fit(X_train, y_train)
```

Why this is correct: scaling and encoding are learned from **training** and applied to **test** without leaking test statistics.

4.3 Step 3: inspect the transformed feature space (optional but useful)

```
Xtr = pipe.named_steps["pre"].transform(X_train)
print("Transformed train shape:", Xtr.shape)
```

If you need column names:

```
feat_names = pipe.named_steps["pre"].get_feature_names_out()
print(feat_names[:20])
```

5 What to write in reports (good explanations)

- "We standardized numeric features to remove scale effects, because our model uses gradients/distances."
- "We used one-hot encoding with `handle_unknown='ignore'` so unseen categories in test do not crash the model."
- "We used a pipeline to avoid leakage: preprocessing parameters were fit using training data only."

6 Troubleshooting

- **Pipeline error:** check column names and ensure `target` exists.
- **Too many one-hot columns:** remove ID-like categorical features; group rare categories.
- **Model performs worse after scaling:** some models (trees) do not need scaling; explain model choice.

Sources: Provost and Fawcett, *Data Science for Business.*;McKinney, *Python for Data Analysis.*;WEKA Documentation (Waikato Environment for Knowledge Analysis), accessed 2026-02-28.;Matplotlib and Seaborn documentation, accessed 2026-02-28.